

Is the Hardware in the Trades?

What actually executes a live trade – and what the RTL and Vivado work really bought

17 June 2026

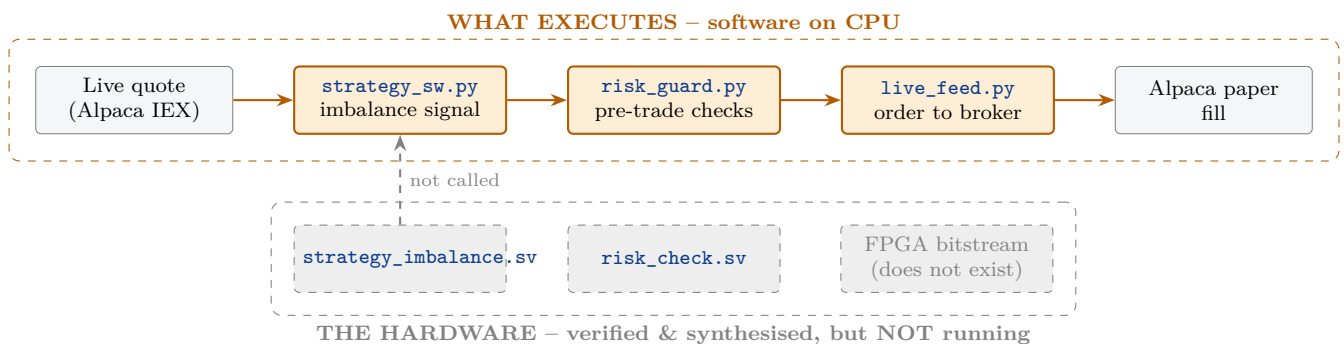
Short answer (the honest truth)

The live paper trades you have run **do NOT use the FPGA hardware**. Every trade was computed by the **Python software model** (`host/strategy_sw.py` + `host/risk_guard.py`), running on your CPU. **No .sv file executes during a trade, no bitstream exists, and no FPGA (local or AWS F1) is in the loop.** The “205 ns” printed next to each signal is a *hard-coded constant* (`FPGA_LATENCY_NS = 205`) copied from a *simulation* result — it is shown for comparison, not measured live.

So why does the project still mean something? Because the software model is *provably identical* to the hardware (bit-for-bit, verified), and the Vivado work *proves the hardware is real and meets 200 MHz timing*. The trades are therefore exactly the decisions the FPGA *would* make — proven by equivalence, not produced by silicon. The remaining step to put real silicon in the loop is the AWS F1 build (Phase 3), which has not yet been done.

1. What Actually Runs During a Live Trade

When you run `python host/live_feed.py --alpaca --execute`, the import list tells the whole story — it pulls in *three Python modules and nothing else*: `strategy_sw`, `risk_guard`, `ibkr_bridge`. There is no FPGA device handle, no PCIe/XDMA driver, no `.bit` or `.awsxclbin` file loaded. The data path is 100% software on the CPU:



Component	In a live trade?	Evidence
<code>strategy_sw.py</code> (Python)	YES	imported & called per quote
<code>risk_guard.py</code> (Python)	YES	checks every order
<code>strategy_imbalance.sv</code> (RTL)	NO	never imported; RTL only runs in QuestaSim
<code>risk_check.sv</code> (RTL)	NO	same
Vivado-implemented design / bitstream	NO	no bitstream built; no FPGA present
The “205 ns” figure	shown, not run	constant <code>FPGA_LATENCY_NS=205</code> from simulation

2. Three Separate Artifacts – and What Each One Is

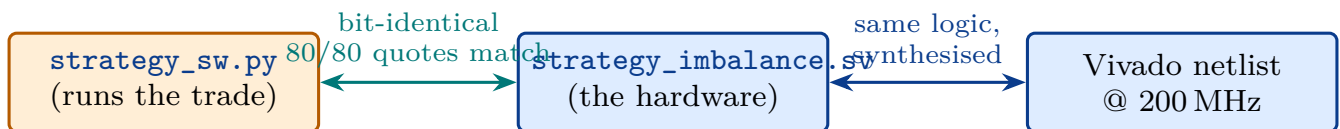
The project produces *three* representations of the same trading logic. They are developed together but they **run in completely different places**. Confusing them is the heart of your question, so here they are side by side:

Artifact	Where it runs	What it proves / does
(A) Python model <code>strategy_sw.py</code> , <code>risk_guard.py</code>	On your CPU, in <code>live_feed.py</code> , on live market data.	Produces the <i>actual</i> trades. This is the only thing that has touched a real (paper) market.
(B) RTL in simulation <code>*.sv</code> in QuestaSim	Inside the QuestaSim simulator on your PC, driven by cocotb testbenches.	Proves the hardware <i>logic</i> is correct (matches the golden model) and reports the 205 ns latency via <code>latency_counter.sv</code> . This is a <i>simulated</i> clock, not silicon.
(C) RTL implemented in Vivado synthesised + placed & routed netlist	Inside Vivado as a timing/area/power analysis. <i>Not</i> downloaded to any chip.	Proves the same <code>.sv</code> is physically <i>realisable</i> : it fits the VU9P, draws known power, and closes timing at 200 MHz . This is what makes the 205 ns <i>credible</i> rather than a guess.

The trades are artifact (A). The “hardware” is artifacts (B) and (C) — both verified, neither *executing* a trade. There is no fourth artifact (a running FPGA) yet; that is the AWS F1 step still to do.

3. How the Hardware Is “Included” – by Equivalence

The hardware is connected to the trades by a **proven equivalence chain**, not by execution. Each link is verified so that “what the software decided” and “what the FPGA would have decided” are guaranteed identical:



- **Software $\equiv$ RTL.** `tb_shadow_quotes.py` feeds the *same* quote stream to `strategy_sw.py` and to the RTL in QuestaSim and diffs every decision: **80/80 quotes matched, 44 software signals = 44 RTL signals, 0 mismatches** (`reports/shadow_report.csv`). The Python model is therefore a faithful stand-in for the gate-level behaviour.
- **RTL $\equiv$ implemented design.** Vivado synthesis + place-and-route turn that exact `.sv` into a gate-level netlist and confirm it behaves identically (functional gate sim, `gatesim/`) while meeting timing.

So when a trade fires, you can correctly say: “*this is the decision the FPGA would produce in 205 ns,*” because (A) $\equiv$ (B) $\equiv$ (C) is proven. What you cannot yet say is “*the FPGA produced this decision*” — because no FPGA ran it.

4. What the Vivado Synthesis & Implementation Bought

It is fair to ask: if the bitstream never ran, what was the point of the Vivado work? It is what separates a *real hardware design* from a Python script that merely *claims* it would be fast. Vivado provided hard, measured evidence:

Vivado output	What it establishes
Timing closure (WNS +0.105 ns, 0 failing endpoints @ 200 MHz)	The design genuinely runs at 200 MHz $\Rightarrow$ the 5 ns clock period is real $\Rightarrow$ 41 cycles <i>is</i> 205 ns. Without this, “205 ns” would be an unproven assumption.
Resource usage (~7,043 LUTs, 4,576 FFs, 1 BRAM, 0 DSP for <code>tick_to_trade_top</code>)	The logic fits the VU9P with vast headroom $\Rightarrow$ it is buildable, and scalable to multi-symbol (~20 k LUTs for 4 symbols).
Power (~2.6 W total)	Realistic, deployable power envelope.
Gate-level netlist (<code>gatesim/</code>)	Post-synthesis functional sim confirms the synthesised gates behave like the source RTL — the logic survives the translation to hardware primitives.

In short: **simulation (B) proves the logic is correct and measures 205 ns; Vivado (C) proves that 205 ns is physically achievable on the real device.** Together they make the latency claim trustworthy. They do not, by themselves, put the chip in the trading loop.

5. The Three Latency Numbers – Don’t Mix Them Up

Number	Source	Meaning
205 ns	RTL in QuestaSim (tb_phase3_pipeline.py) + Vivado timing	Simulated tick-to-trade of the hardware, validated as physically achievable. <i>Not</i> measured on silicon.
~9 μs	Live run, time.perf_counter_ns around strategy_sw.evaluate	The <i>actual</i> software compute latency on your CPU during real trades — the only one measured live.
~tens of ms	Network round-trip to the broker	The dominant real-world delay on the live order path.

The headline “ $\sim 45\times$ faster on hardware” compares 205 ns (B/C) to the $\approx 9\mu$ s software compute (the live measurement) — an *apples-to-apples compute* comparison of the same logic on two substrates.

6. What Would Make the FPGA Actually Execute Trades

To move from “the FPGA *would* decide this” to “the FPGA *did* decide this,” the remaining work (Phase 3, all gated on AWS) is:

1. Build an **AFI** (Amazon FPGA Image) from [f1/cl_tick_to_trade.sv](#) on an EC2 FPGA Developer instance (full Vivado Shell build).
2. Launch an **f1.2xlarge** and load the AFI onto the VU9P.
3. DMA/stream ITCH into the device and read decisions + the real latency histogram back via [f1/host_replay.py --device](#).
4. Diff the silicon decisions against the golden model and confirm **≈ 205 ns on real hardware**.

Only after step 4 is the 205 ns a *silicon-measured* number and the FPGA genuinely in the decision path. The local clone has **no bitstream** today, which is why none of the live trades touched hardware.